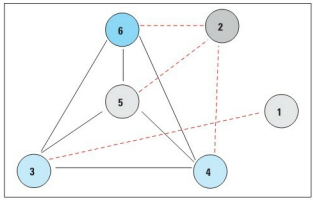

Figure 8: ILP restrictions graph (IRG)

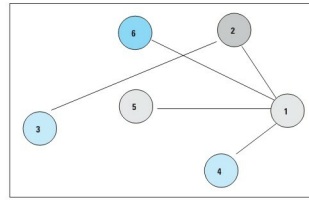

Figure 9: ILP graph

- Instruction 1 and Instruction 3
- Instruction 2 and Instruction 4
- Instruction 2 and Instruction 5
- Instruction 2 and Instruction 6

Thus, we add four new edges to the undirected TCG, to obtain the ILP Restriction Graph as shown in Figure 8. The architecture restriction edges are dashed lines shown in red.

Step #4: Construct the ILP graph

The ILP Graph (see Figure 9) shows all the possibilities of parallelism between instructions, and is the **complement** of the ILP Restrictions Graph obtained in Step 3.

The ILP graph shows that we have possibilities of parallelism between the following instruction pairs:

- Instruction 1 and Instruction 2
- Instruction 1 and Instruction 4
- Instruction 1 and Instruction 5
- Instruction 1 and Instruction 6
- Instruction 2 and Instruction 3

Step #5: Schedule and apply ILP

In the schedule and apply ILP step, we try to achieve the parallelism depicted in the ILP graph. Assume that our hypothetical architecture supports ILP of a maximum of two instructions in parallel. As we saw above, Instruction 1 shows the possibility of parallelism with any of the following instructions—2, 4, 5 or 6. So which instruction should we pair with Instruction 1?

The selection of an ILP pair instruction is an important step, because it can have an impact on any further ILPs to be achieved. Let's explore this practically with our subject code block.

Case A: Let's attempt to combine Instructions 1 and 2 in parallel:

```
1. MOVE  R0, [AR0];      ADD   R1, R2, R2;
3. MOVE  R3, [AR1];
4. ADD   R4, R3, R1;
5. ADD   R3, R4, R4;
6. SUB R3, R3, R4
```

The result is that no other ILP is possible now in the instruction set, because we have already utilised Instruction 2, which was a candidate for parallelism with Instruction 3. Now the ILP of Instructions 2 and 3 (as was shown by the ILP graph) can no longer be achieved. So, this case is a bad selection of instructions for parallelism.

Case B: We select Instruction 1 to be paired with Instruction 4:

```
1. MOVE  R0, [AR0]       ADD   R4, R3, R1;
2. ADD   R1, R2, R2;
3. MOVE  R3, [AR1];
5. ADD   R3, R4, R4;
6. SUB R3, R3, R4
```

Now, we can easily achieve the parallelism of Instructions 2 and 3, as well:

```
1. MOVE  R0, [AR0]       ADD   R4, R3, R1;
2. ADD   R1, R2, R2      MOVE  R3, [AR1];
5. ADD   R3, R4, R4;
6. SUB R3, R3, R4
```

The selection of instructions for parallelism is a critical step. The idea is to select the instructions in such a way that the maximum ILPs are achieved. Since our goal is to reduce the number of instruction cycles as well as the code size, the more the ILPs we achieve, the better. Since Case B clearly yields more ILPs than Case A, this is our chosen case to maximise ILP. END

References and further reading

- Narsingh Deo: *Graph Theory with Applications to Engineering and Computer Science.*
- Sima, Fountain and Kacsuk: *Advanced Computer Architectures - A Design Space Approach.*

By: Deepti Sharma

The author is a senior software engineer (Team Lead) at Acme Technologies. She has a considerable amount of experience in the development of compilers and tools for embedded systems. She can be reached at deepti.acmet@gmail.com.